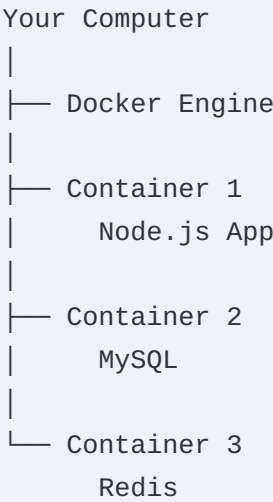


חלק 1 – יסודות Docker (קונטיינרים)

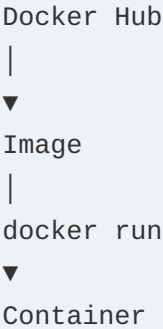
מה זה Docker?

Docker (ספריות, Python, Node.js) היא פלטפורמה המאפשרת לארוז אפליקציה יחד עם כל מה שהיא צריכה Docker קבצי מערכת הפעלה וכו') לתוך קונטיינר (Container). תחשבו על זה כך:



כל קונטיינר מבודד לחלוטין מהאחרים.

ארכיטקטורת Docker



- Docker Hub ← מאגר מקוון של אימג'ים (Images)
- Image ← תבנית / שרטוט (Blueprint)
- Container ← מופע רץ של אימג' (Instance)

דוגמה:

```
ubuntu image
|
```

```
docker run
|
▼
ubuntu container
```

ניתן ליצור קונטיינרים רבים מאותו אימג'.

פקודות בסיסיות

בדיקת גרסת Docker

```
docker --version
```

דוגמה: Docker version 28.2.2

הצגת מידע על Docker

```
docker info
```

מציג מידע על: קונטיינרים רצים, אימג'ים, אחסון, מעבדים (CPUs) וזיכרון.

הורדת אימג'

```
docker pull nginx
```

פקודה זו מורידה את האימג' העדכני ביותר של nginx. להורדת גרסה ספציפית:

```
docker pull nginx:1.27
```

אם לא צוינה גרסה, תורד ברירת המחדל - `latest`.

רשימת אימג'ים

```
docker images
```

הפעלת קונטיינר

```
docker run nginx
```

יבצע את הפעולות הבאות Docker:

1. יחפש את האימג' מקומית.
2. יוריד אותו אם הוא חסר.
3. ייצור קונטיינר.

| מציבי ריצה (Run Modes)

מצב אינטראקטיבי (Interactive Mode) - דוגמה עם Ubuntu

```
docker run -it ubuntu
```

בתוך הקונטיינר ניתן להריץ פקודות כגון `ls`, `pwd`, `apt update`, ליציאה, הקלד `exit`.

מצב מנותק (Detached Mode)

במקום שהמסוף "ייתפס" (Attached) לקונטיינר, נריץ אותו ברקע באמצעות הדגל `-d`:

```
docker run -d nginx
```

| ניהול קונטיינרים

רשימת קונטיינרים

רק קונטיינרים רצים:

```
docker ps
```

כל הקונטיינרים (כולל כאלו שנעצרו):

```
docker ps -a
```

עצירה, התחלה והסרה

- עצירת קונטיינר: `docker stop container_name` (לדוגמה: `docker stop 95df34`)
- התחלת קונטיינר קיים: `docker start container_name`
- הפעלה מחדש: `docker restart container_name`
- הסרת קונטיינר: `docker rm container_name` (חייב להיות כבוי)
- הסרה בכוח: `docker rm -f container_name`

| הגדרות רשת וסביבה

מיפוי פורטים (Port Mapping)

בלי מיפוי, המחשב המארח (Host) לא יכול לגשת לפורט של הקונטיינר. בעזרת `-p`:

```
docker run -p 8080:80 nginx
```

המשמעות: פורט 8080 במארח מנותב לפורט 80 בקונטיינר. עכשיו ניתן לגשת ל- `http://localhost:8080`.

אפשר גם להגדיר מספר פורטים:

```
docker run -p 3000:3000 -p 9229:9229 image
```

משתני סביבה (Environment Variables)

הרבה אימג'ים דורשים הגדרות. נעשה זאת בעזרת `-e`:

```
docker run -e MYSQL_ROOT_PASSWORD=123456 mysql
```

מתן שם לקונטיינר

במקום לקבל שמות אקראיים, נשתמש ב- `--name`:

```
docker run --name web nginx
```

| גישה וניטור

הרצת פקודות בתוך קונטיינר רץ

```
docker exec -it web bash
```

צפייה בלוגים

```
docker logs web
```

מעקב מתמשך (Follow): `docker logs -f web`

הסרת אימג'ים

```
docker rmi nginx
```

אפשרות	תיאור
-d	מצב מנותק (Detached mode) - הרצה ברקע
-it	מסוף אינטראקטיבי
-p host:container	מיפוי פורטים
-e KEY=VALUE	משתני סביבה
--name	שם הקונטיינר
--rm	מחיקת הקונטיינר באופן אוטומטי ביציאה

| דוגמה מקיפה – הרצת מסד נתונים MySQL

1. משיכת האימג': `docker pull mysql:8.4`
2. הפעלת הקונטיינר:

```
docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456
mysql:8.4
```

3. וידוא ריצה: `docker ps`
4. צפייה בלוגים: `docker logs -f mysql-db` (יש להמתין להודעה שהמסד נתונים מוכן).
5. פתיחת Shell בקונטיינר: `docker exec -it mysql-db bash`
6. התחברות ללקוח MySQL ישירות:

```
docker exec -it mysql-db mysql -u root -p
```

פקודות SQL בסיסיות לתרגול

```
SHOW DATABASES; CREATE DATABASE school; USE school; CREATE TABLE students (id INT
AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100)); INSERT INTO students (name) VALUES
('Moshe'); SELECT * FROM students; exit
```

| תהליך עבודה מלא (Workflow)

```
Pull Image
|
▼
docker pull mysql:8.4
|
```



Run Container

|



```
docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456 mysql:8.4
```

|



List Running Containers

|



```
docker ps
```

|



View Logs

|



```
docker logs -f mysql-db
```

|



Open Linux Shell

|



```
docker exec -it mysql-db bash
```

|



Open MySQL Client

|



```
docker exec -it mysql-db mysql -u root -p
```